UNIT – II

(Part –I Software Project Management Renaissance)

Conventional Software Management

1. The best thing about software is its flexibility:

- It can be programmed to do almost anything.

2. The worst thing about software is its flexibility:

- The “almost anything” characteristic has made it difficult to plan, monitor, and control software development.

3. In the mid-1990s, three important analyses were performed on the software engineering industry.

All three analyses given the same general conclusion:-

“The success rate for software

projects is very low”. They Summarized as follows:

1. Software development is still highly unpredictable. Only 10% of software projects are delivered successfully within initial budget and scheduled time.
2. Management discipline is more differentiator in success or failure than are technology advances.
3. The level of software scrap and rework is indicative of an immature process.

WATERFALLL MODEL

1. It is the baseline process for most conventional software projects have used.
2. We can examine this model in two ways:

i. IN THEORY

ii. IN PRACTICE

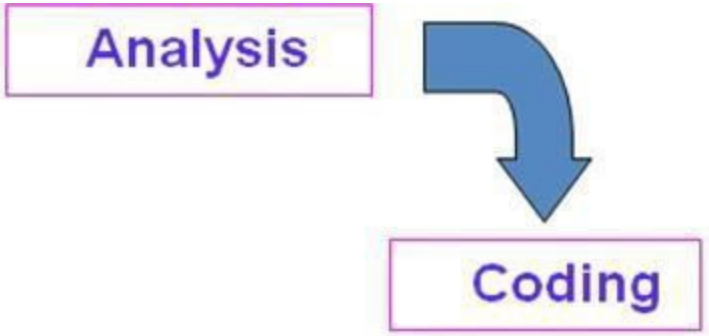
IN THEORY:-

In 1970, Winston Royce presented a paper called "Managing the Development of Large Scale Software Systems at IEEE WESCON.

Where he made three primary points:

1. There are two essential steps common to the development of computer programs:

- analysis
- Coding



Analysis and coding both involve creative work that directly contributes to the usefulness of the end product

Waterfall Model Part 1: The two basic steps to build a program

2. In order to mange and control all of the intellectual freedom associated with software

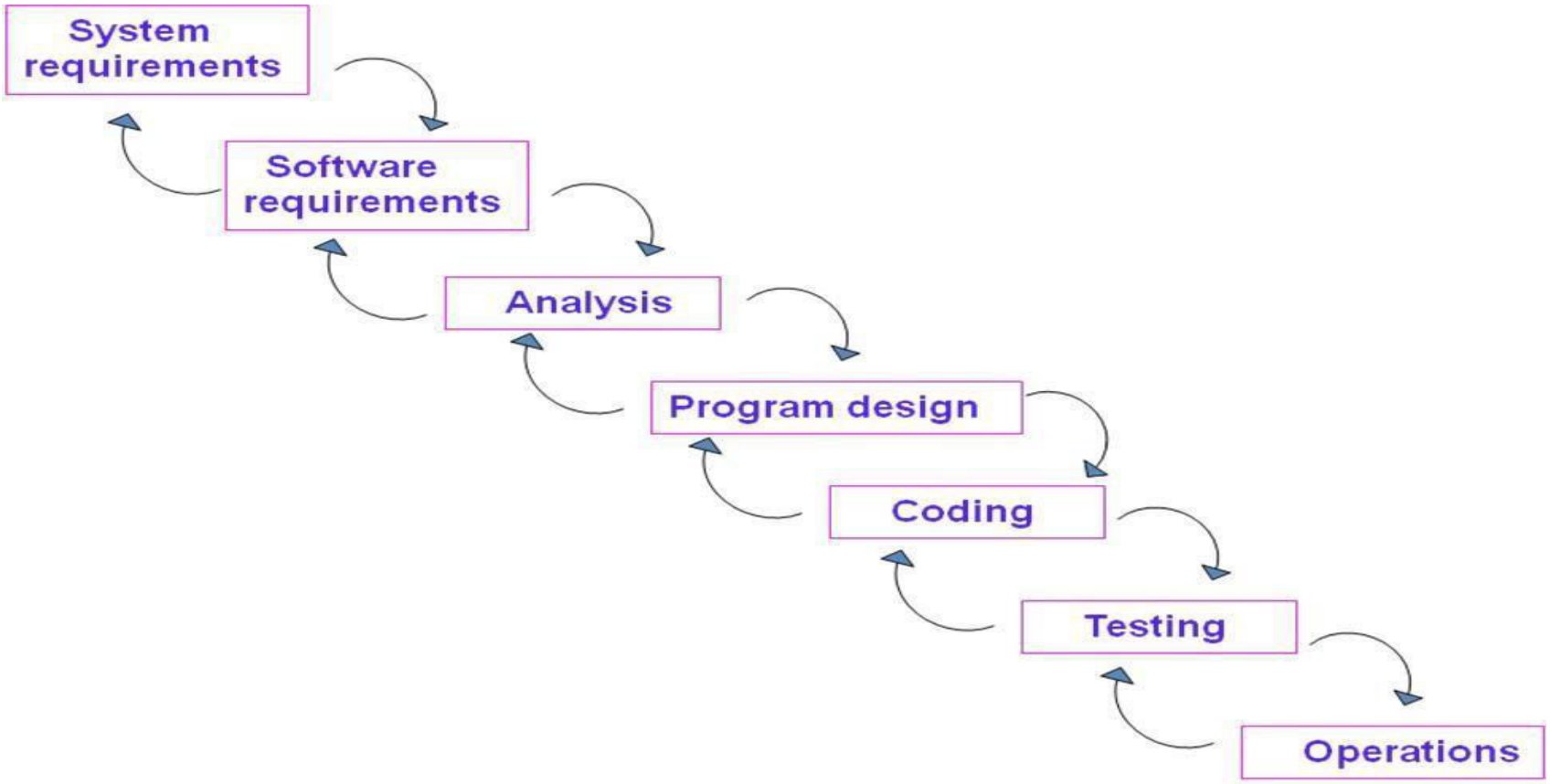
development one should follow the following steps:

-System requirements definition

-Software requirements definition

- Program design and testing

These steps addition to the analysis and coding steps



Waterfall Model Part 2: The large – scale system approach

3. Since the testing phase is at the end of the development cycle in the waterfall model, it may be risky and invites failure.

So we need to do either the requirements must be modified or a substantial design changes is warranted by breaking the software in to different pieces.

-There are five improvements to the basic waterfall model that would eliminate most of the development risks

are as follows:

a) Complete program design before analysis and coding begin (program design comes first):-

- By this technique, the program designer give surety that the software will not fail because of storage, timing, and

data fluctuations.

- Begin the design process with program designer, not the analyst or programmers.

- Write an overview document that is understandable, informative, and current so that every worker on the project

can gain an elemental understanding of the system.

b) Maintain current and complete documentation (Document the design):-

-It is necessary to provide a lot of documentation on most sw programs

c) Do the job twice, if possible (Do it twice):-

- If a computer program is developed for the first time, arrange matters so that the version finally delivered to the customer for operational deployment is actually the second version insofar as critical design/operations are

concerned.

- "Do it N times" approach is the principle of modern-day iterative development.

d) Plan, control, and monitor testing:-

- The biggest user of project resources is the test phase. This is the phase of greatest risk in terms of cost and schedule.
- In order to carryout proper testing the following things to be done:

i) Employ a team of test specialists who were not responsible for the original design.

ii) Employ visual inspections to spot the obvious errors like dropped minus signs, missing factors of two, jumps to wrong addresses.

iii) Test every logic phase.

iv) Employ the final checkout on the target computer.

e) Involve the customer:-

- It is important to involve the customer in a formal way so that he has committed himself at earlier points before final delivery by conducting some reviews such

i) Preliminary software review during preliminary program design step.

ii) Critical software review during program design.

iii) Final software acceptance review following testing

IN PRACTICE:-

- Whatever the advices that are given by the software developers and the theory behind the waterfall model, some software projects still practice the conventional software management approach.

Projects intended for trouble frequently exhibit the following symptoms:

i) Protracted (delayed) integration

- In the conventional model, the entire system was designed on paper, then implemented all at once, then integrated. Only at the end of this process was it possible to perform system testing to verify that the fundamental architecture was sound.

Here the testing activities consume 40% or more life-cycle resources.

ACTIVITY COST

Management 5%

Requirements 5%

Design 10%

Code and unit testing 30%

Integration and test 40%

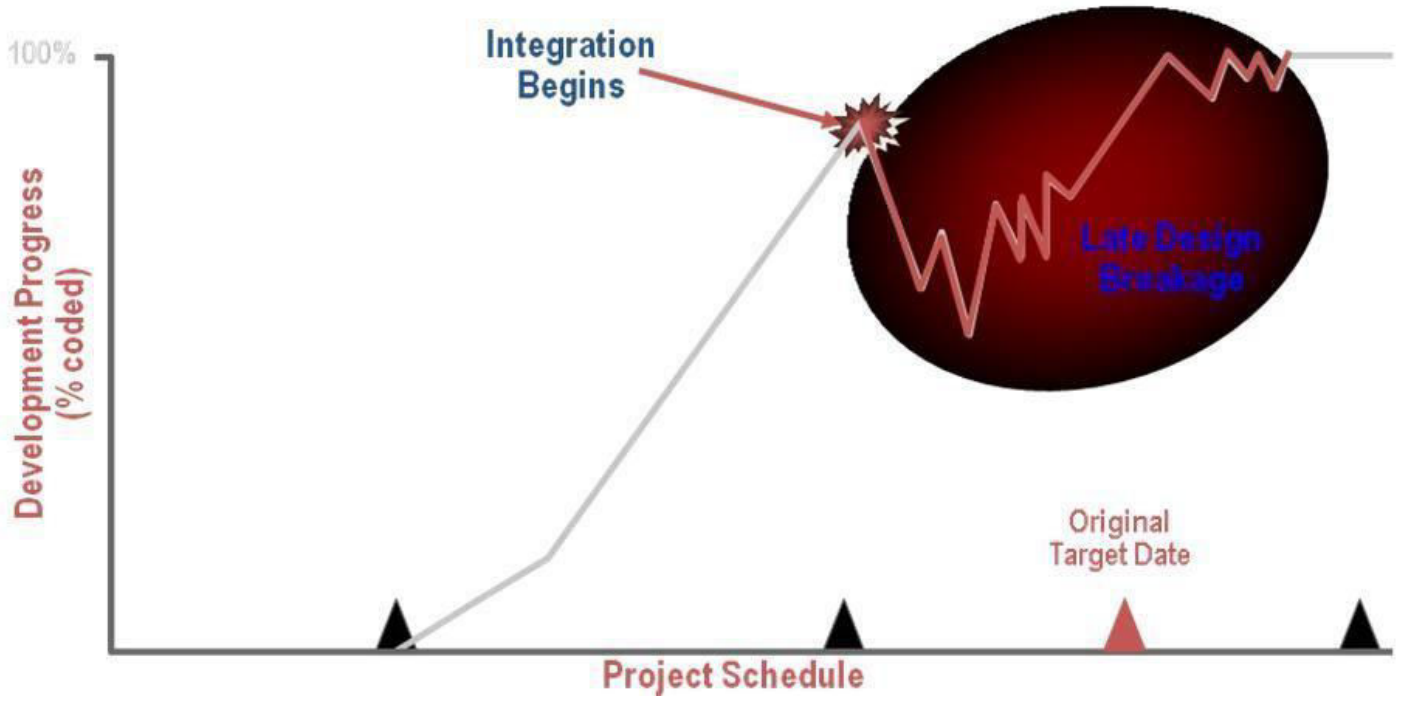
Deployment 5%

Environment 5%

What Happens in Practice

Sequential activities:

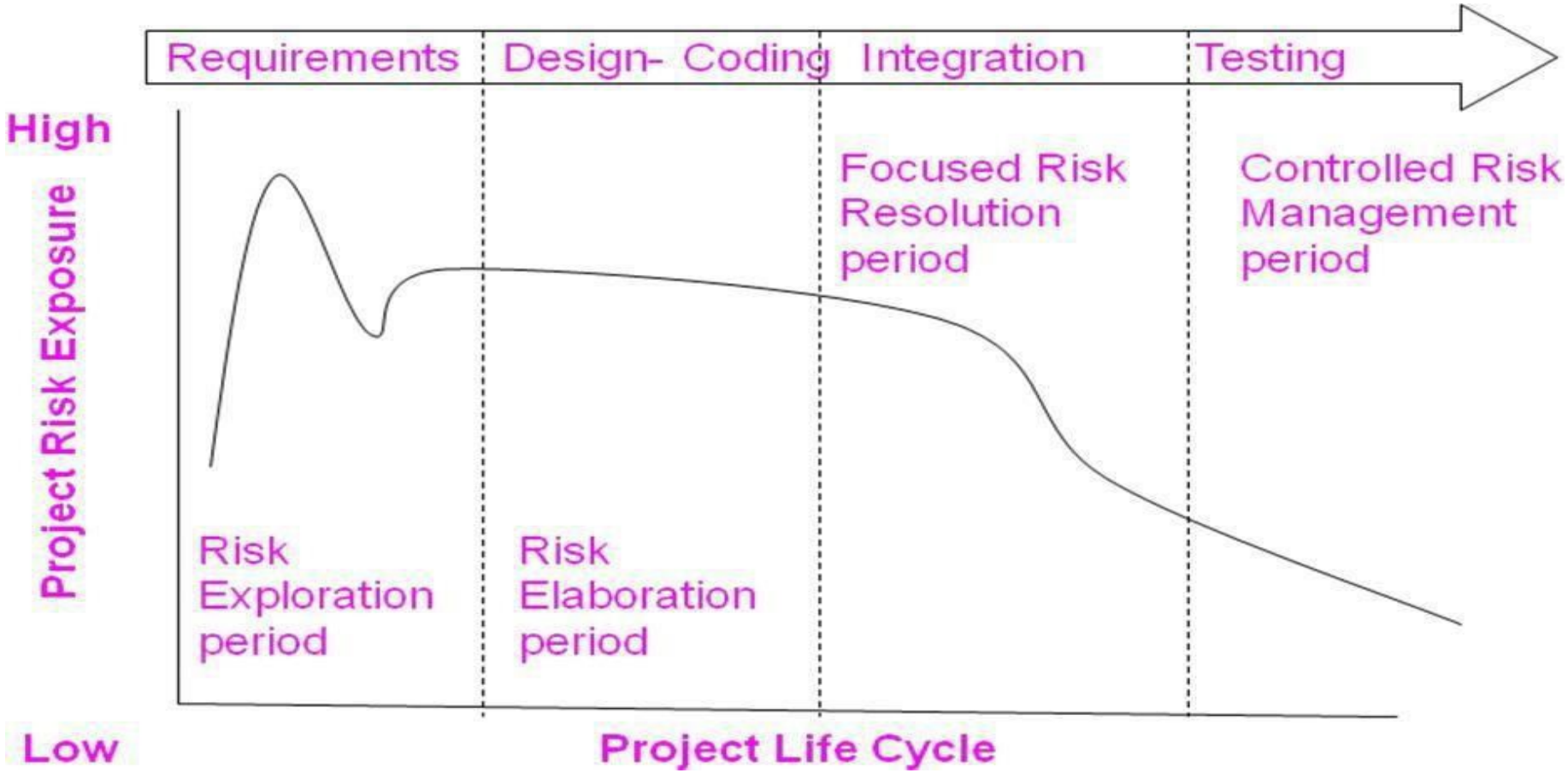
Requirements → Design → Code → Integration → Test



ii) Late Risk Resolution

- A serious issues associated with the waterfall life cycle was the lack of early risk resolution.

The risk profile of a waterfall model is,



iii) Requirements-Driven Functional Decomposition

-Traditionally, the software development process has been requirement-driven: An attempt is made to provide a precise requirements definition and then to implement exactly those requirements.

-This approach depends on specifying requirements completely and clearly before other development activities begin.

iv) Adversarial Stakeholder Relationships

The following sequence of events was typical for most contractual software efforts:

- The contractor prepared a draft contact-deliverable document that captured an intermediate artifact and delivered it to the customer for approval.

- The customer was expected to provide comments (within 15 to 30 days)

Project Stakeholders :

Stakeholders are the people involved in or affected by project activities

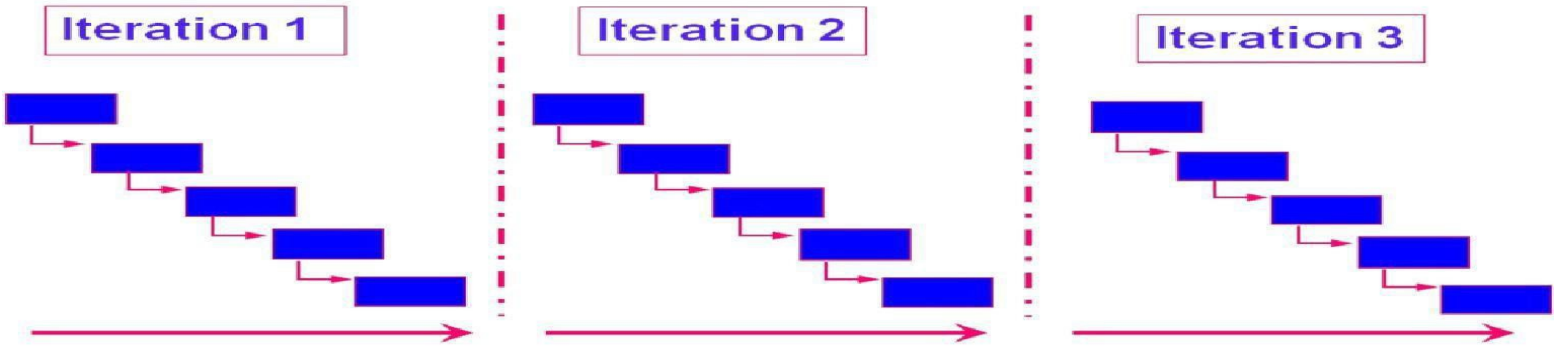
Stakeholders include

- the project sponsor and project team
- support staff
- customers
- users
- suppliers
- opponents to the project

v) Focus on Documents and Review Meetings

- The conventional process focused on various documents that attempted to describe the software product.
- Contractors produce literally tons of paper to meet milestones and demonstrate progress to stakeholders, rather than spend their energy on tasks that would reduce risk and produce quality software.
- Most design reviews resulted in low engineering and high cost in terms of the effort and schedule involved in their preparation and conduct.

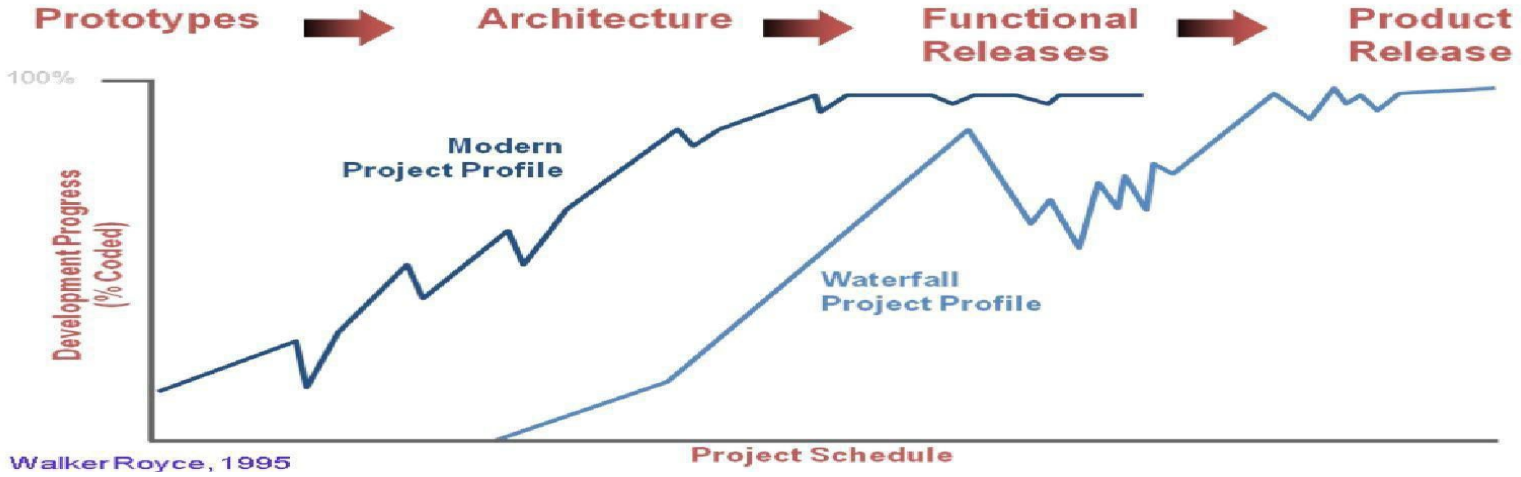
Iterative Development



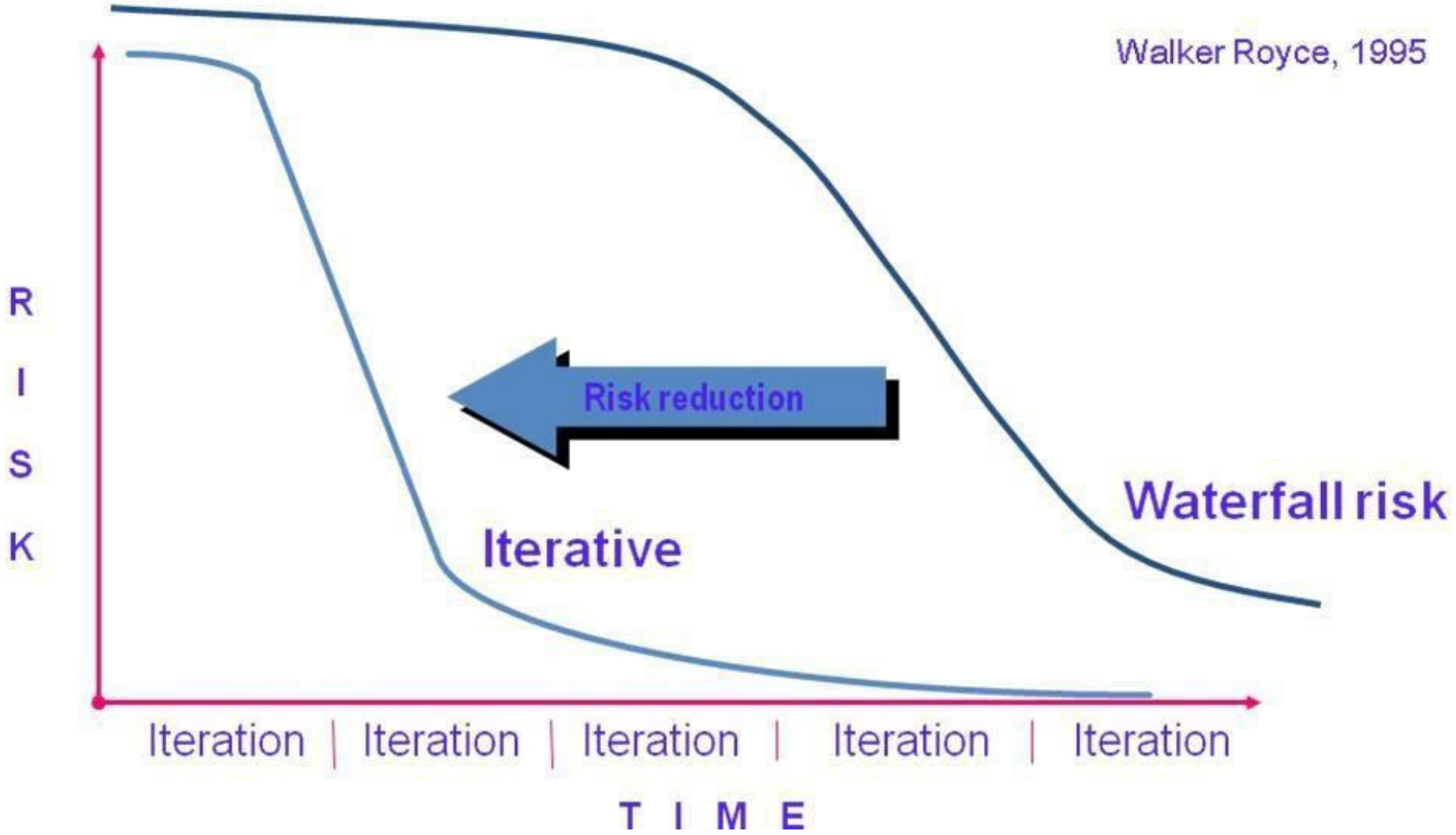
- Earliest iterations address greatest risks
- Each iteration produces an executable release
- Each iteration includes integration and test

Better Progress Profile

Sequential phases, but iterative activities



Accelerate Risk Reduction



Iterative Development Phases



Inception: Agreement on overall scope
- Vision, high-level requirements, business case
- Not detailed requirements

Elaboration: Agreement on design approach
- Baseline architecture, most requirements detailed
- Not detailed design

Construction: Apply approach
- Working product, system test complete

Transition: Validate solution
- Stakeholder acceptance

Benefits of Iterative Development

- **Risk mitigation**
  - Requirements, architecture, technology, organization and process
- **Accommodating changes**
  - Functional changes, tactical changes, technology
- **Learning as you go**
  - Process improvement, feedback loop
  - Individual learning
- **Better opportunity for reuse**
  - Reuse often identified too late to be exploited
- **Greater quality**
  - Tested more and more often
- **Greater predictability, minimize losses**
  - Earlier failure or cancellation

Barry Boehm‟s Top 10 “Industrial Software Metrics”:

1) Finding and fixing a software problem after delivery costs 100 times more than finding and fixing the problem in early design phases.

2) You can compress software development schedules 25% of nominal (small), but no more.

3) For every $1 you spend on development, you will spend $2 on maintenance.

4) Software development and maintenance costs are primarily a function of the number of source lines of code.

5) Variations among people account for the biggest difference in software productivity.

6) The overall ratio of software to hardware costs is still growing. In 1955 it was 15:85; in 1985, 85:15.

7) Only about 15% of software development effort is devoted to programming.

8) Software systems and products typically cost 3 times as much per SLOC as individual software programs. Software-system products cost 9 times as much.

9) Walkthroughs catch 60% of the errors.

10) 80% of the contribution comes from 20% of the contributors.

- 80% of the engineering is consumed by 20% of the requirements.
- 80% of the software cost is consumed by 20% of the components.
- 80% of the errors are caused by 20% of the components.
- 80% of the software scrap and rework is caused by 20% of the errors.
- 80% of the resources are consumed by 20% of the components.
- 80% of the engineering is accomplished by 20% of the tools.
- 80% of the progress is made by 20% of the people.

In the following paragraphs, quotations from Boehm's top 10 list are presented in *italics*, followed by my comments.

1. *Finding and fixing a software problem after delivery costs 100 times more than finding and fixing the problem in early design phases.*

   ▲ This metric dominates the rationale for virtually every dimension of process improvement discussed in this or any other book. It is not unique to software development. When one of the big automobile companies implements a recall for a post-delivery defect, the cost of repair can be many orders of magnitude greater than the cost of fixing the defect during the engineering or production stage.

2. *You can compress software development schedules 25% of nominal, but no more.*

   ▲ One reason for this is that an N% reduction in schedule would require an M% increase in personnel resources (assuming that other parameters remain fixed). Any increase in the number of people requires more management overhead. In general, the limit of flexibility in this overhead, along with scheduling concurrent activities, conserving sequential activities, and other resource constraints, is about 25%. Optimally, a 100-staff-month effort may be achievable in 10 months by 10 people. Could the job be done in one month with 100 people? Two months with 50 people? How about 5 months with 20 people? Clearly, these alternatives are unrealistic. The 25% compression metric says that the limit in this case is 7.5 months (and would require additional staff-months, perhaps as many as 20). Any further schedule compression is doomed to fail. On the other hand, an optimal schedule could be extended almost arbitrarily and, depending on the people, could be performed in a much longer time with many fewer staff resources. For example, if you have the luxury of a 25-month schedule, you may need only 75 staff-months and three people.

3. *For every $1 you spend on development, you will spend $2 on maintenance.*

   ▲ Boehm calls this the "iron law of software development." Whether you build a long-lived product that undergoes commercial version upgrades twice a year or build a one-of-a-kind custom software system, twice as much money will probably be spent over the maintenance life cycle than was spent in the development life cycle. It is hard to tell at first whether this relationship is good or bad. In the commercial product domain, the primary driver of this relationship is the product's success in the marketplace. Successful software products (such as Oracle, Microsoft applications, Rational Rose, and the UNIX operating system) are very long lived and can result in much higher ratios of maintenance cost to development cost. Managers of one-of-a-kind software projects, on the other hand, rarely plan to expend this much on software maintenance. In either case, anyone working in the software industry over the past 10 to 20 years knows that most of the software in operation is considered to be difficult to maintain.

4. *Software development and maintenance costs are primarily a function of the number of source lines of code.*

   ▲ This metric is primarily the result of the predominance of custom software development, lack of commercial componentry, and lack of reuse inherent in the era of the conventional process.

5. *Variations among people account for the biggest differences in software productivity.*

   ▲ This is a key piece of conventional wisdom: Hire good people. This metric is a subject of both overhype and underhype. When you don't know objectively why you succeeded or failed, the obvious scapegoat is the quality of the people. This judgment is subjective and difficult to challenge.

6. *The overall ratio of software to hardware costs is still growing. In 1955 it was 15:85; in 1985, 85:15.*

   ▲ The fact that software represents 85% of the cost of most systems is not so much a statement about software productivity (which is, arguably, not as good as we want) as it is about the level of functionality being allocated to software in system solutions. The need for software, its breadth of applications, and its complexity continue to grow almost without limits.

7. *Only about 15% of software development effort is devoted to programming.*

   ▲ This is an important indicator of the need for balance. Many activities besides coding are necessary for software project success. Requirements management, design, testing, planning, project control, change management, and toolsmithing are equally important considerations that consume roughly 85% of the resources.

8. *Software systems and products typically cost 3 times as much per SLOC as individual software programs. Software-system products (i.e., system of systems) cost 9 times as much.*

   ▲ This exponential relationship is the essence of what is called *diseconomy of scale.* Unlike other commodities, the more software you build, the more expensive it is per source line.

9. *Walkthroughs catch 60% of the errors.*

   ▲ This may be true. However, given metric 1, walkthroughs are not catching the errors that matter and certainly are not catching them early enough in the life cycle. All defects are not created equal. In general, walkthroughs and other forms of human inspection are good at catching surface problems and style issues. If you are using ad hoc design notations, human review may be your primary quality assurance

mechanism, but it is not good at uncovering second-, third-, and Nth-order issues such as resource contention, performance bottlenecks, control conflicts, and so on. Furthermore, few humans are good at reviewing even first-order semantic issues in a code segment. How many programmers get their code to compile the first time?

10. *80% of the contribution comes from 20% of the contributors.*

▲ This is a motherhood statement that is true across almost any engineering discipline (or any professional discipline, for that matter). I have expanded this metric into a more specific interpretation for software. The following fundamental postulates underlie the rationale for a modern software management process framework:

80% of the engineering is consumed by 20% of the requirements.

80% of the software cost is consumed by 20% of the components.

80% of the errors are caused by 20% of the components.

80% of software scrap and rework is caused by 20% of the errors.

80% of the resources are consumed by 20% of the components.

80% of the engineering is accomplished by 20% of the tools.

80% of the progress is made by 20% of the people.

These relationships provide some good benchmarks for evaluating process improvements and technology improvements. They represent rough rules of thumb that objectively characterize the performance of the conventional software management process and conventional technologies. In later chapters, I return to many of these measures to rationalize a new approach, defend an old approach, and quantify process or technology improvements.

Evolution of Software Economics

Economics means System of interrelationship of money, industry and employment.

SOFTWAR ECONOMICS:-

The cost of the software can be estimated by considering the following things as parameters to a function.

1) Size: Which is measured in terms of the number of Source Lines Of Code or the

number of function points required to develop the required functionality.

2) Process: Used to produce the end product, in particular the ability of the process is to avoid

non-value-adding activities (rework, bureaucratic delays, communications overhead).

3) Personnel: The capabilities of software engineering personnel, and particularly their

experience with the computer science issues and the application domain issues of the project.

4) Environment: Which is made up of the tools and techniques available to support

efficient software development and to automate the process.

5) Quality: It includes its features, performance, reliability, and flexibility.

The relationship among these parameters and estimated cost can be calculated by using,

- Effort =(Personnel) (Environment) (Quality) ($Size^{Process}$)

A performance measure used to evaluate the efficiency of an investment or to compare the efficiency of a number of different investments. To calculate ROI, the benefit (return) of an investment is divided by the cost of the investment; the result is expressed as a percentage or a ratio.

The return on investment formula:

$$ROI = \frac{(\text{Gain from Investment} - \text{Cost of Investment})}{\text{Cost of Investment}}$$

Project Sizes :

- Size as team strength could be :
  - Trivial (Minor) Size: 1 person
  - Small Size: 5 people
  - Moderate Size: 25 people
  - Large Size: 125 people
  - Huge Size: 625 people
- The more the size, the greater are the costs of management overhead, communication, synchronizations among various projects or modules, etc.

Reduce Software Size:

The less software we write, the better it is for project management and for product quality -

The cost of software is not just in the cost of „coding“ alone; it is also in

Analysis of requirements

– Design

– Review of requirements, design and code

– Test Planning and preparation

– Testing

– Bug fix

– Regression testing

– „Coding“ takes around 15% of development cost

- Clearly, if we reduce 15 hrs of coding, we can directly reduce 100 hrs of development effort, and also reduce the project team size appropriately!

PRAGMATIC SOFTWARE ESTIMATION:

- If there is no proper well-documented case studies then it is difficult to estimate the cost of the

software. It is one of the critical problem in software cost estimation.

- But the cost model vendors claim that their tools are well suitable for estimating iterative development projects.

- In order to estimate the cost of a project the following three topics should be considered,

1) Which cost estimation model to use.

2) Whether to measure software size in SLOC or function point.

3) What constitutes a good estimate.

- There are a lot of software cost estimation models are available such as, COCOMO,

CHECKPOINT, ESTIMACS, Knowledge Plan, Price-S,

ProQMS, SEER, SLIM, SOFTCOST, and SPQR/20.

- Of which COCOMO is one of the most open and well-documented cost estimation models

- The software size can be measured by using

1) SLOC 2) Function points

Topic 3:Improving Software Economics

- It is not that much easy to improve the software economics but also difficult to measure and validate.
- There are many aspects are there in order to improve the software economics they are, Size, Process, Personnel, Environment and quality.
- These parameters (aspects) are not independent they are dependent. For example, tools enable size reduction and process improvements, size-reduction approaches lead to process changes, and process improvements drive tool requirements.

REDUCING SOFTWARE PRODUCT SIZE:

- By choosing the type of the language
- By using Object-Oriented methods and visual modeling
- By reusing the existing components and building reusable components &
- By using commercial components,we can reduce the product size of a software

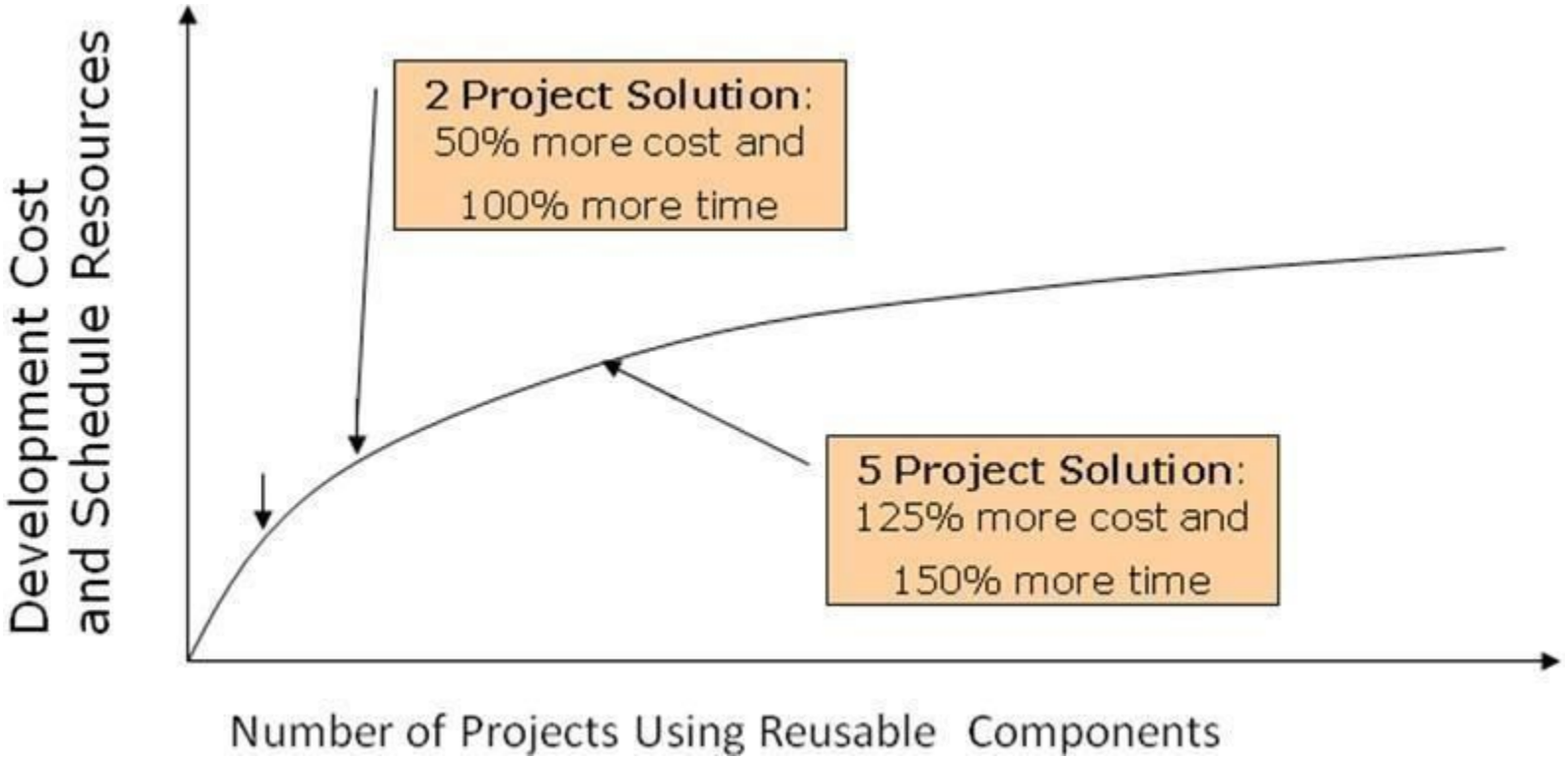
Here UPFs (Universal Function Points) are useful estimators for language- independent in the early life cycle phases. The basic units of function points are:

- External user inputs
- External outputs
- Internal logical data groups
- External data Interfaces
- External inquiries

***REUSE*:**

Organizations that translates reusable components into commercial products has the following characteristics:

- They have an economic motivation for continued support.
- They take ownership of improving product quality, adding new features and transitioning to new technologies.
- They have a sufficiently broad customer base to be profitable.


Development Cost
and Schedule Resources
2 Project Solution:
50% more cost and
100% more time
5 Project Solution:
125% more cost and
150% more time
Number of Projects Using Reusable Components

IMPROVING SOFTWARE PROCESSES:

There are three distinct process perspectives:

- **Meta process:**

It is an Organization‟s policies, procedures, and practices for pursuing a software-intensive line of business.

The focus of this process is of organizational economics, long-term strategies, and a software ROI.

- **Macro process:**

A project‟s policies, and practices for producing a complete software product within certain cost, schedule, and quality constraints.

The focus of the macroprocess is on creating an sufficient instance of the metaprocess for a specific set of constraints.

- **Micro process:**

A projects team‟s policies, procedures, and practices for achieving an artifact of a software process.

The focus of the microprocess is on achieving an intermediate product baseline with sufficient functionality as economically and rapidly as practical.

IMPROVING TEAM EFFECTIVENESS:

- COCOMO model suggests that the combined effects of personnel skill and experience can have an impact on productivity as much as a factor of four over the unskilled personnel.
- Balance and coverage are two of the most important features of excellent teams. Whenever a team is in out of balance then it is vulnerable.
- It is the responsibility of the project manager to keep track of his teams. Since teamwork is much more important than the sum of the individuals.

Boehm – staffing principles:

- **The principle of top talent: Use better and fewer people.**
- **The principle of job matching: Fit the tasks to the skills and motivation of the**

people available.

- **The principle of career progression: An organization does best in the long run by**

helping its people to self-actualize.

4) **The principle of team balance: Select people who will complement and**

synchronize with one another.

5) **The principle of phase-out: Keeping a misfit on the team doesn‟t benefit anyone.**

In general, staffing is achieved by these common methods:

– If people are already available with required skill set, just take them

– If people are already available but do not have the required skills, re-train them

– If people are not available, recruit trained people

– If you are not able to recruit skilled people, recruit and train people

Staffing of key personnel is very important:

- Project Manager

- Software Architect

Important Project Manager Skills:

Hiring skills

Customer-interface skill

Decision-making skill.

Team-building skill.

Selling skill

ACHIEVING REQUIRED QUALITY:

Key elements that improve overall software quality include the following:

- Focusing on powerful requirements and critical use case early in the life cycle
- Focusing on requirements completeness and traceability late in the life cycle
- Focusing throughout the life cycle on a balance between requirements evolution, design evolution, and plan evolution
- Using metrics and indicators to measure the progress and quality of an architecture as it evolves from high-level prototype into a fully biddable product
- Providing integrated life-cycle environments that support early and continuous configuration control, change management, rigorous design methods, document automation, and regression test automation
- Using visual modeling and higher level languages that support architectural control,abstraction, reliable programming, reuse, and self-documentation
- Early and continuous close look into performance issues through demonstration-based evaluations

In order to evaluate the performance the following sequence of events are necessary,

1) Project inception 2) Initial design review

2) Mid-life-cycle design review 4) Integration and test

Topic 3:THE OLD WAY AND THE NEW

- Over the past two decades software development is a re-engineering process. Now

it is replaced by advanced software engineering technologies.

- This transition is was motivated by the unsatisfactory demand for the software and reduced cost.

THE PRINCIPLES OF CONVENTIONAL SOFTWARE ENGINEERING

Based on many years of software development experience, the software industry proposed so many principles (nearly 201 by – Davis‟s). Of which Davis‟s top 30 principles are:

1) **Make quality #1: Quality must be quantified and mechanisms put into place** to motivate its achievement.

2) **High-quality software is possible: In order to improve the quality of the** product we need to involving the customer, select the prototyping, simplifying design, conducting inspections, and hiring the best people.

1) **Make quality #1: Quality must be quantified and mechanisms put into place**

to motivate its achievement.

2) **High-quality software is possible: In order to improve the quality of the**

product we need to involving the customer, select the prototyping, simplifying

design, conducting inspections, and hiring the best people.

3) **Give products to customers early: No matter how hard you try to learn user‟s** needs during the requirements phase, the most effective way to determine real needs is to give users a product and let them play with it.

4) **Determine the problem before writing the requirements: Whenever a** problem is raised most engineers provide a solution. Before we try to solve a problem, be sure to explore all the alternatives and don‟t be blinded by the understandable solution.

5) Evaluate design alternatives: After the requirements are greed upon, we must examine a variety of architectures and algorithms and choose the one which is not used earlier.

6) **Use an appropriate process model: For every project, there are so many**

prototypes (process models). So select the best one that is exactly suitable to our project.

7) **Use different languages for different phases: Our industry‟s main aim is to** provide simple solutions to complex problems. In order to accomplish this goal choose different languages for different modules/phases if required.

8) **Minimize intellectual distance: We have to design the structure of a software** is as close as possible to the real-world structure.

9) **Put techniques before tools: An un disciplined software engineer with a tool** becomes a dangerous, undisciplined software engineer.

10) **Get it right before you make it faster: It is very easy to make a working** program run faster than it is to make a fast program work. Don‟t worry about optimization during initial coding.

11) Inspect the code: Examine the detailed design and code is a much better way to find the errors than testing.

12) **Good management is more important than good technology**

13) People are the key to success: Highly skilled people with appropriate experience, talent, and training are key. The right people with insufficient tools, languages, and process will succeed.

14) **Follow with care: Everybody is doing something but does not make it right**

for you. It may be right, but you must carefully assess its applicability to your environment.

15) **Take responsibility: When a bridge collapses we ask "what did the engineer do** wrong?". Similarly if the software fails, we ask the same. So the fact is in every engineering discipline, the best methods can be used to produce poor results and the most out of date methods to produce stylish design.

16) **U nderstand the customer's priorities . It is possible the customer would** tolerate 90% of the functionality delivered late if they could have 10% of it on

time.

18) **Plan to throw one away .One of the most important critical success factors is** whether or not a product is entirely new. Such brand-new applications, architectures, interfaces, or algorithms rarely work the first time.

19) **Design for change. The architectures, components, and** specification techniques you use must accommodate change.

20) **Design without documentation is not design. I have often heard software** engineers say, “I have finished the design. All that is left is the documentation.”

THE PRINCIPLES OF MODERN SOFTWARE MANAGEMENT

1) Base the process on an architecture-first approach: (Central design element)

- Design and integration first, then production and test

2) Establish an iterative life-cycle process: (The risk management element)

- Risk control through ever-increasing function, performance, quality.

3) Transition design methods to emphasize component-based development: (The technology element)

Moving from LOC mentally to component-based mentally is necessary to reduce the amount of human-generated source code and custom development.A component is

a cohesive set of preexisting lines of code, either in source or executable format, with a defined interface and behavior.

4) Establish a change management environment: (The control element)

- Metrics, trends, process instrumentation

The dynamics of iterative development, include concurrent workflows by different teams working on shared artifacts, necessitates objectively controlled baseline.

5) Enhance change freedom through tools that support round-trip engineering: (The

automation element)

- Complementary tools, integrated environment

Round-trip engineering is the environment support necessary to automate and synchronize engineering information in different formats. Change freedom is necessary in an iterative process.

6) Capture design artifacts in rigorous, model-based notation:

- A model-based approach supports the evolution of semantically rich graphical and textual design notations.

- Visual modeling with rigorous notations and formal machine-process able language provides more objective measures than the traditional approach of human review and inspection of ad hoc design representations in paper doc.

7) Instrument the process for objective quality control and progress assessment:

- Life-cycle assessment of the progress and quality of all intermediate product must be

integrated into the process.

- The best assessment mechanisms are well-defined measures derived directly from the

evolving engineering artifacts and integrated into all activities and teams.

8) Use a demonstration-based approach to assess intermediate artifacts:

Transitioning from whether the artifact is an early prototype, a baseline

architecture, or a beta capability into an executable demonstration of relevant provides more

tangible understanding of the design tradeoffs, early integration and earlier elimination of

architectural defects.

9) Plan intermediate releases in groups of usage scenarios with

evolving levels of detail:

10) Establish a configurable process that economically scalable:

Architecture-first approach → The central design element

Design and integration first, then production and test

Iterative life-cycle process → The risk management element

Risk control through ever-increasing function, performance, quality

Component-based development → The technology element

Object-oriented methods, rigorous notations, visual modeling

Change management environment → The control element

Metrics, trends, process instrumentation

Round-trip engineering → The automation element

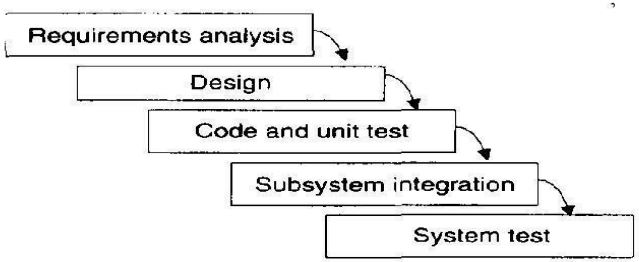
Complementary tools, integrated environments

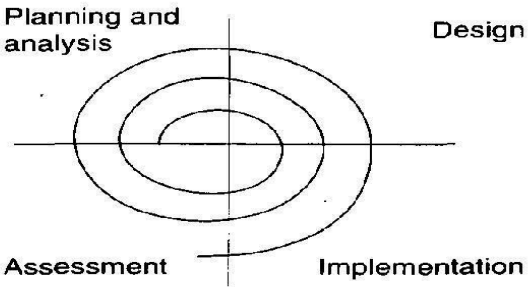
Waterfall Process

Requirements first
Custom development
Change avoidance
Ad hoc tools

Iterative Process

Architecture first
Component-based development
Change management
Round-trip engineering





Transitioning to an Iterative process

Transitioning to a Modern Process

- Modern process features –
  - Early development of an initial version.
  - High risk areas are addressed early.
  - Several iterations are developed (called spirals, increments, generations, releases).
- Modern process characteristics-
  - Extensive use of domain experience.
  - Process flexibility and change management.
  - Architecture risk resolution.
  - Team cohesion.
  - Software process maturity.

UNIT – II

The old way and the new: The principles of conventional software Engineering, principles of modern software management, transitioning to an iterative process.

Life cycle phases: Engineering and production stages, inception, Elaboration, construction, transition phases.

Artifacts of the process: The artifact sets, Management artifacts, Engineering artifacts, programmatic artifacts.

4. THE OLD WAY AND THE NEW

4.1 THE PRINCIPLES OF CONVENTIONAL SOFTWARE ENGINEERING

1.**Make quality #1.** Quality must be quantified and mechanisms put into place to motivate its achievement

2.**High-quality software is possible**. Techniques that have been demonstrated to increase quality include involving the customer, prototyping, simplifying design, conducting inspections, and hiring the best people

3.**Give products to customers early**. No matter how hard you try to learn users' needs during the requirements phase, the most effective way to determine real needs is to give users a product and let them play with it

4.**Determine the problem before writing the requirements**. When faced with what they believe is a problem, most engineers rush to offer a solution. Before you try to solve a problem, be sure to explore all the alternatives and don't be blinded by the obvious solution

5.**Evaluate design alternatives**. After the requirements are agreed upon, you must examine a variety of architectures and algorithms. You certainly do not want to use" architecture" simply because it was used in the requirements specification.

6.**Use an appropriate process model**. Each project must select a process that makes ·the most sense for that project on the basis of corporate culture, willingness to take risks, application area, volatility of requirements, and the extent to which requirements are well understood.

7.**Use different languages for different phases**. Our industry's eternal thirst for simple solutions to complex problems has driven many to declare that the best development method is one that uses the same notation throughout the life cycle.

8.**Minimize intellectual distance**. To minimize intellectual distance, the software's structure should be as close as possible to the real-world structure

9.**Put techniques before tools**. An undisciplined software engineer with a tool becomes a dangerous, undisciplined software engineer

10.**Get it right before you make it faster**. It is far easier to make a working program run faster than it is to make a fast program work. Don't worry about optimization during initial coding

11.**Inspect code**. Inspecting the detailed design and code is a much better way to find errors than testing

12.**Good management is more important than good technology**. Good management motivates people to do their best, but there are no universal "right" styles of management.

13.**People are the key to success.** Highly skilled people with appropriate experience, talent, and training are key.

14.**Follow with care**. Just because everybody is doing something does not make it right for you. It may be right, but you must carefully assess its applicability to your environment.

15.**Take responsibility**. When a bridge collapses we ask, "What did the engineers do wrong?" Even when software fails, we rarely ask this. The fact is that in any engineering discipline, the best methods can be used to produce awful designs, and the most antiquated methods to produce elegant designs.

16.**Understand the customer's priorities**. It is possible the customer would tolerate 90% of the functionality delivered late if they could have 10% of it on time.

17.**The more they see, the more they need**. The more functionality (or performance) you provide a user, the more functionality (or performance) the user wants.

18. **Plan to throw one away**. One of the most important critical success factors is whether or not a product is entirely new. Such brand-new applications, architectures, interfaces, or algorithms rarely work the first time.

19. **Design for change**. The architectures, components, and specification techniques you use must

accommodate change.

20. **Design without documentation is not design**. I have often heard software engineers say, **"I** have finished the design. All that is left is the documentation. "
21. **Use tools, but be realistic.** Software tools make their users more efficient.
22. **Avoid tricks**. Many programmers love to create programs with tricks constructs that perform a function correctly, but in an obscure way. Show the world how smart you are by avoiding tricky code
23. **Encapsulate.** Information-hiding is a simple, proven concept that results in software that is easier to test and much easier to maintain.
24. **Use coupling and cohesion**. Coupling and cohesion are the best ways to measure software's inherent maintainability and adaptability
25. **Use the McCabe complexity measure**. Although there are many metrics available to report the inherent complexity of software, none is as intuitive and easy to use as Tom McCabe's
26. **Don't test your own software**. Software developers should never be the primary testers of their own software.
27. **Analyze causes for errors**. It is far more cost-effective to reduce the effect of an error by preventing it than it is to find and fix it. One way to do this is to analyze the causes of errors as they are detected
28. **Realize that software's entropy increases**. Any software system that undergoes continuous change will grow in complexity and will become more and more disorganized
29. **People and time are not interchangeable**. Measuring a project solely by person-months makes little sense
30. **Expect excellence**. Your employees will do much better if you have high expectations for them.

4.2 THE PRINCIPLES OF MODERN SOFTWARE MANAGEMENT

Top 10 principles of modern software management are. (The first five, which are the main themes of my definition of an iterative process, are summarized in Figure 4-1.)

Base the process on an *architecture-first approach.* This requires that a demonstrable balance be achieved among the driving requirements, the architecturally significant design decisions, and the life-cycle plans before the resources are committed for full-scale development.

Establish an *iterative life-cycle process* that confronts risk early. With today's sophisticated software systems, it is not possible to define the entire problem, design the entire solution, build the software, and then test the end product in sequence. Instead, an iterative process that refines the problem understanding, an effective solution, and an effective plan over several iterations encourages a balanced treatment of all stakeholder objectives. Major risks must be addressed early to increase predictability and avoid expensive downstream scrap and rework.

Transition design methods to emphasize *component-based development.* Moving from a line-of-code mentality to a component-based mentality is necessary to reduce the amount of human-generated source code and custom development.

4. **Establish a *change management environment.*** The dynamics of iterative development, including concurrent workflows by different teams working on shared artifacts, necessitates objectively controlled baselines.

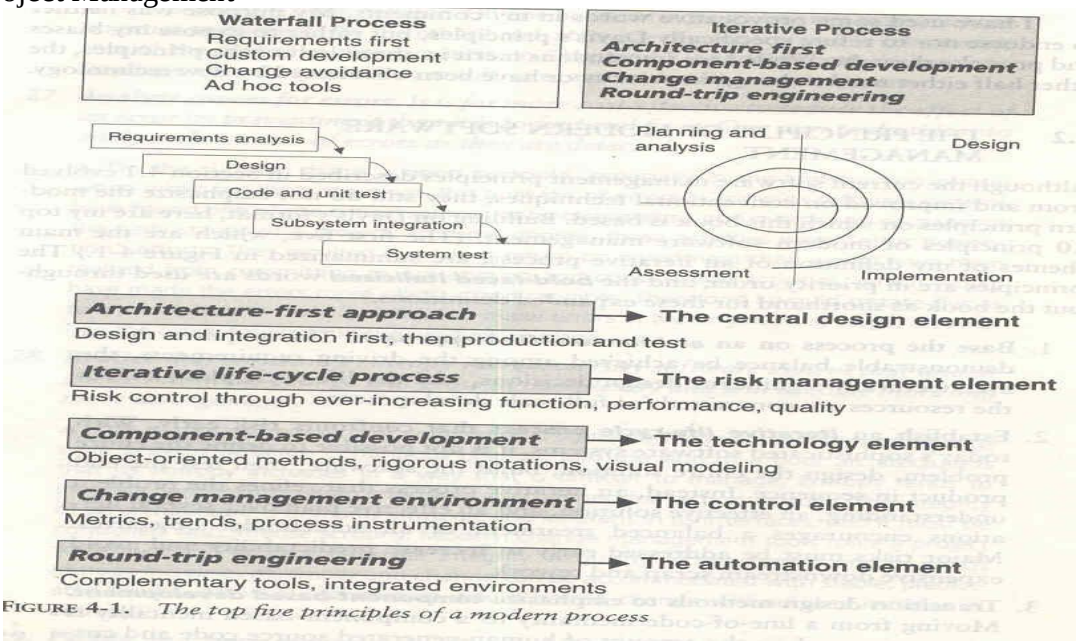


FIGURE 4-1. *The top five principles of a modern process*

5. **Enhance change freedom through tools that support round-trip engineering**. Round-trip engineering is the environment support necessary to automate and synchronize engineering information in different formats(such as requirements specifications, design models, source code, executable code, test cases).
6. **Capture design artifacts in rigorous, model-based notation.** A model based approach (such as UML) supports the evolution of semantically rich graphical and textual design notations.
7. **Instrument the process for objective quality control and progress assessment**. Life-cycle assessment of the progress and the quality of all intermediate products must be integrated into the process.
8. **Use a demonstration-based approach to assess intermediate artifacts.**
9. **Plan intermediate releases in groups of usage scenarios with evolving levels of detail.** It is essential that the software management process drive toward early and continuous demonstrations within the operational context of the system, namely its use cases.
10. **Establish a configurable process that is economically scalable.** No single process is suitable for all software developments.

Table 4-1 maps top 10 risks of the conventional process to the key attributes and principles of a modern process

TABLE 4-1. *Modern process approaches for solving conventional problems*

CONVENTIONAL PROCESS: TOP 10 RISKS	IMPACT	MODERN PROCESS: INHERENT RISK RESOLUTION FEATURES
1. Late breakage and excessive scrap/rework	Quality, cost, schedule	Architecture-first approach Iterative development Automated change management Risk-confronting process
2. Attrition of key personnel	Quality, cost, schedule	Successful, early iterations Trustworthy management and planning
3. Inadequate development resources	Cost, schedule	Environments as first-class artifacts of the process Industrial-strength, integrated environments Model-based engineering artifacts Round-trip engineering
4. Adversarial stakeholders	Cost, schedule	Demonstration-based review Use-case-oriented requirements/testing
5. Necessary technology insertion	Cost, schedule	Architecture-first approach Component-based development
6. Requirements creep	Cost, schedule	Iterative development Use case modeling Demonstration-based review
7. Analysis paralysis	Schedule	Demonstration-based review Use-case-oriented requirements/testing
8. Inadequate performance	Quality	Demonstration-based performance assessment Early architecture performance feedback
9. Overemphasis on artifacts	Schedule	Demonstration-based assessment Objective quality control
10. Inadequate function	Quality	Iterative development Early prototypes, incremental releases

4.3 TRANSITIONING TO AN ITERATIVE PROCESS

Modern software development processes have moved away from the conventional waterfall model, in which each stage of the development process is dependent on completion of the previous stage.

The economic benefits inherent in transitioning from the conventional waterfall model to an iterative development process are significant but difficult to quantify. As one benchmark of the expected economic impact of process improvement, consider the process exponent parameters of the COCOMO II model. (Appendix B provides more detail on the COCOMO model) This exponent can range from 1.01 (virtually no diseconomy of scale) to 1.26 (significant diseconomy of scale). The parameters that govern the value of the process exponent are application precedentedness, process flexibility, architecture risk resolution, team cohesion, and software process maturity.

The following paragraphs map the process exponent parameters of CO COMO II to my top 10 principles of a modern process.

- **Application precedentedness**. Domain experience is a critical factor in understanding how to plan and execute a software development project. For unprecedented systems, one of the key goals is to confront risks and establish early precedents, even if they are incomplete or experimental. This is one of the primary reasons that the software industry has moved to an ***iterative life-cycle process.*** Early iterations in the life cycle establish precedents from which the product, the process, and the plans can be elaborated in ***evolving levels* of *detail.***
- **Process flexibility**. Development of modern software is characterized by such a broad solution space and so many interrelated concerns that there is a paramount need for continuous incorporation of changes. These changes may be inherent in the problem understanding, the solution space, or the plans. Project artifacts must be supported by efficient ***change management*** commensurate with project needs. A ***configurable process*** that allows a common framework to be adapted across a range of projects is necessary to achieve a software return on investment.
- **Architecture risk resolution**. ***Architecture-first*** development is a crucial theme underlying a successful iterative development process. A project team develops and stabilizes architecture before

developing all the components that make up the entire suite of applications components. An ***architecture-first*** and ***component-based development approach*** forces the infrastructure, common mechanisms, and control mechanisms to be elaborated early in the life cycle and drives all component make/buy decisions into the architecture process.

- **Team cohesion**. Successful teams are cohesive, and cohesive teams are successful. Successful teams and cohesive teams share common objectives and priorities. Advances in technology (such as programming languages, UML, and visual modeling) have enabled more rigorous and understandable notations for communicating software engineering information, particularly in the requirements and design artifacts that previously were ad hoc and based completely on paper exchange. These ***model-based*** formats have also enabled the ***round-trip engineering*** support needed to establish change freedom sufficient for evolving design representations.
- **Software process maturity**. The Software Engineering Institute's Capability Maturity Model (CMM) is a well-accepted benchmark for software process assessment. One of key themes is that truly mature processes are enabled through an integrated environment that provides the appropriate level of automation to instrument the process for ***objective quality control.***

Important questions

1.	***Explain briefly Waterfall model. Also explain Conventional s/w management performance?***
2.	***Define Software Economics. Also explain Pragmatic s/w cost estimation?***
3.	***Explain Important trends in improving Software economics?***
4.	***Explain five staffing principal offered by Boehm. Also explain Peer Inspections?***
5..	***Explain principles of conventional software engineering?***
6.	***Explain briefly principles of modern software management***

5. Life cycle phases

Characteristic of a successful software development process is the well-defined separation between "research and development" activities and "production" activities. Most unsuccessful projects exhibit one of the following characteristics:

- An overemphasis on research and development
- An overemphasis on production.

Successful modern projects-and even successful projects developed under the conventional process-tend to have a very well-defined project milestone when there is a noticeable transition from a research attitude to a production attitude. Earlier phases focus on achieving functionality. Later phases revolve around achieving a product that can be shipped to a customer, with explicit attention to robustness, performance, and finish.

A modern software development process must be defined to support the following:

- Evolution of the plans, requirements, and architecture, together with well defined synchronization points
- Risk management and objective measures of progress and quality
- Evolution of system capabilities through demonstrations of increasing functionality

5.1 ENGINEERING AND PRODUCTION STAGES

To achieve economies of scale and higher returns on investment, we must move toward a software manufacturing process driven by technological improvements in process automation and component-based development. Two stages of the life cycle are:

1. The **engineering stage**, driven by less predictable but smaller teams doing design and synthesis activities
2. The **production stage**, driven by more predictable but larger teams doing construction, test, and deployment activities

TABLE 5-1. *The two stages of the life cycle: engineering and production*

LIFE-CYCLE ASPECT	ENGINEERING STAGE EMPHASIS	PRODUCTION STAGE EMPHASIS
Risk reduction	Schedule, technical feasibility	Cost
Products	Architecture baseline	Product release baselines
Activities	Analysis, design, planning	Implementation, testing
Assessment	Demonstration, inspection, analysis	Testing
Economics	Resolving diseconomies of scale	Exploiting economies of scale
Management	Planning	Operations

The transition between engineering and production is a crucial event for the various stakeholders. The production plan has been agreed upon, and there is a good enough understanding of the problem and the solution that all stakeholders can make a firm commitment to go ahead with production.

Engineering stage is decomposed into two distinct phases, inception and elaboration, and the production stage into construction and transition. These four phases of the life-cycle process are loosely mapped to the conceptual framework of the spiral model as shown in Figure 5-1

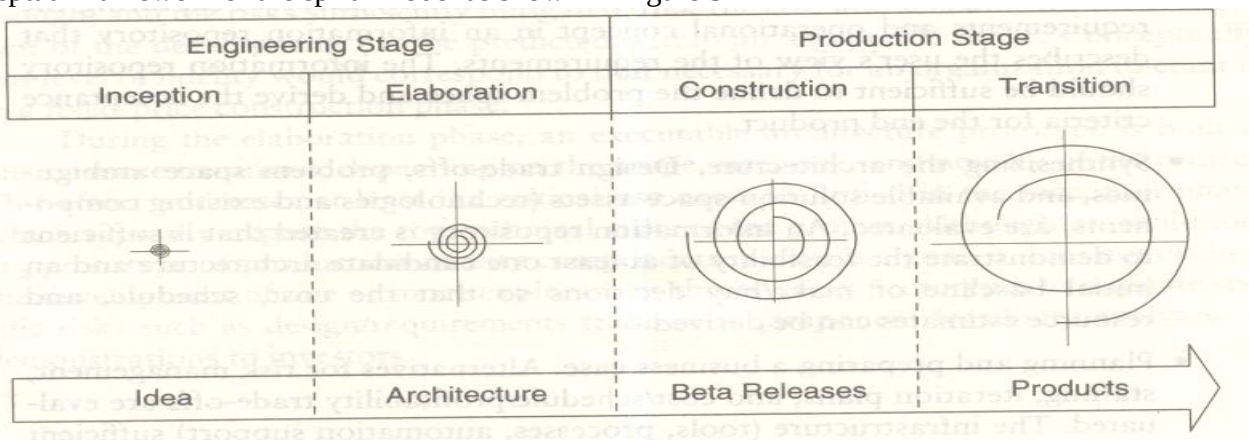


FIGURE 5-1. *The phases of the life-cycle process*

5.2 INCEPTION PHASE

The overriding goal of the inception phase is to achieve concurrence among stakeholders on the life-cycle objectives for the project.

PRIMARY OBJECTIVES

- Establishing the project's software scope and boundary conditions, including an operational concept, acceptance criteria, and a clear understanding of what is and is not intended to be in the product
- Discriminating the critical use cases of the system and the primary scenarios of operation that will drive the major design trade-offs
- Demonstrating at least one candidate architecture against some of the primary scenanos
- Estimating the cost and schedule for the entire project (including detailed estimates for the elaboration phase)
- Estimating potential risks (sources of unpredictability)

ESSENTIAL ACTMTIES

- Formulating the scope of the project. The information repository should be sufficient to define the problem space and derive the acceptance criteria for the end product.
- Synthesizing the architecture. An information repository is created that is sufficient to demonstrate the feasibility of at least one candidate architecture and an, initial baseline of make/buy decisions so that the cost, schedule, and resource estimates can be derived.
- Planning and preparing a business case. Alternatives for risk management, staffing, iteration plans, and cost/schedule/profitability trade-offs are evaluated.

PRIMARY EVALUATION CRITERIA

- Do all stakeholders concur on the scope definition and cost and schedule estimates?
- Are requirements understood, as evidenced by the fidelity of the critical use cases?
- Are the cost and schedule estimates, priorities, risks, and development processes credible?
- Do the depth and breadth of an architecture prototype demonstrate the preceding criteria? (The primary value of prototyping candidate architecture is to provide a vehicle for understanding the scope and assessing the credibility of the development group in solving the particular technical problem.)
- Are actual resource expenditures versus planned expenditures acceptable

5.2 ELABORATION PHASE

At the end of this phase, the "engineering" is considered complete. The elaboration phase activities must ensure that the architecture, requirements, and plans are stable enough, and the risks sufficiently mitigated, that the cost and schedule for the completion of the development can be predicted within an acceptable range. During the elaboration phase, an executable architecture prototype is built in one or more iterations, depending on the scope, size, & risk.

PRIMARY OBJECTIVES

- Baselining the architecture as rapidly as practical (establishing a configuration-managed snapshot in which all changes are rationalized, tracked, and maintained)
- Baselining the vision
- Baselining a high-fidelity plan for the construction phase
- Demonstrating that the baseline architecture will support the vision at a reasonable cost in a reasonable time

ESSENTIAL ACTIVITIES

- Elaborating the vision.

- Elaborating the process and infrastructure.
- Elaborating the architecture and selecting components.

PRIMARY EVALUATION CRITERIA

- Is the vision stable?
- Is the architecture stable?
- Does the executable demonstration show that the major risk elements have been addressed and credibly resolved?
- Is the construction phase plan of sufficient fidelity, and is it backed up with a credible basis of estimate?
- Do all stakeholders agree that the current vision can be met if the current plan is executed to develop the complete system in the context of the current architecture?
- Are actual resource expenditures versus planned expenditures acceptable?

5.4 CONSTRUCTION PHASE

During the construction phase, all remaining components and application features are integrated into the application, and all features are thoroughly tested. Newly developed software is integrated where required. The construction phase represents a production process, in which emphasis is placed on managing resources and controlling operations to optimize costs, schedules, and quality.

PRIMARY OBJECTIVES

- Minimizing development costs by optimizing resources and avoiding unnecessary scrap and rework
- Achieving adequate quality as rapidly as practical
- Achieving useful versions (alpha, beta, and other test releases) as rapidly as practical

ESSENTIAL ACTIVITIES

- Resource management, control, and process optimization
- Complete component development and testing against evaluation criteria
- Assessment of product releases against acceptance criteria of the vision

PRIMARY EVALUATION CRITERIA

- Is this product baseline mature enough to be deployed in the user community? (Existing defects are not obstacles to achieving the purpose of the next release.)
- Is this product baseline stable enough to be deployed in the user community? (Pending changes are not obstacles to achieving the purpose of the next release.)
- Are the stakeholders ready for transition to the user community?
- Are actual resource expenditures versus planned expenditures acceptable?

5.5 TRANSITION PHASE

The transition phase is entered when a baseline is mature enough to be deployed in the end-user domain. This typically requires that a usable subset of the system has been achieved with acceptable quality levels and user documentation so that transition to the user will provide positive results. This phase could include

any of the following activities:

1. Beta testing to validate the new system against user expectations
2. Beta testing and parallel operation relative to a legacy system it is replacing
3. Conversion of operational databases
4. Training of users and maintainers

The transition phase concludes when the deployment baseline has achieved the complete vision.

PRIMARY OBJECTIVES

- Achieving user self-supportability
- Achieving stakeholder concurrence that deployment baselines are complete and consistent with the evaluation criteria of the vision
- Achieving final product baselines as rapidly and cost-effectively as practical

ESSENTIAL ACTIVITIES

- Synchronization and integration of concurrent construction increments into consistent deployment baselines
- Deployment-specific engineering (cutover, commercial packaging and production, sales rollout kit development, field personnel training)
- Assessment of deployment baselines against the complete vision and acceptance criteria in the requirements set

EVALUATION CRITERIA

- Is the user satisfied?
- Are actual resource expenditures versus planned expenditures acceptable?

6. Artifacts of the process

6.1 THE ARTIFACT SETS

To make the development of a complete software system manageable, distinct collections of information are organized into artifact sets. A*rtifact* represents cohesive information that typically is developed and reviewed as a single entity.

Life-cycle software artifacts are organized into five distinct sets that are roughly partitioned by the underlying language of the set: management (ad hoc textual formats), requirements (organized text and models of the problem space), design (models of the solution space), implementation (human-readable programming language and associated source files), and deployment (machine-process able languages and associated files). The artifact sets are shown in Figure 6-1.

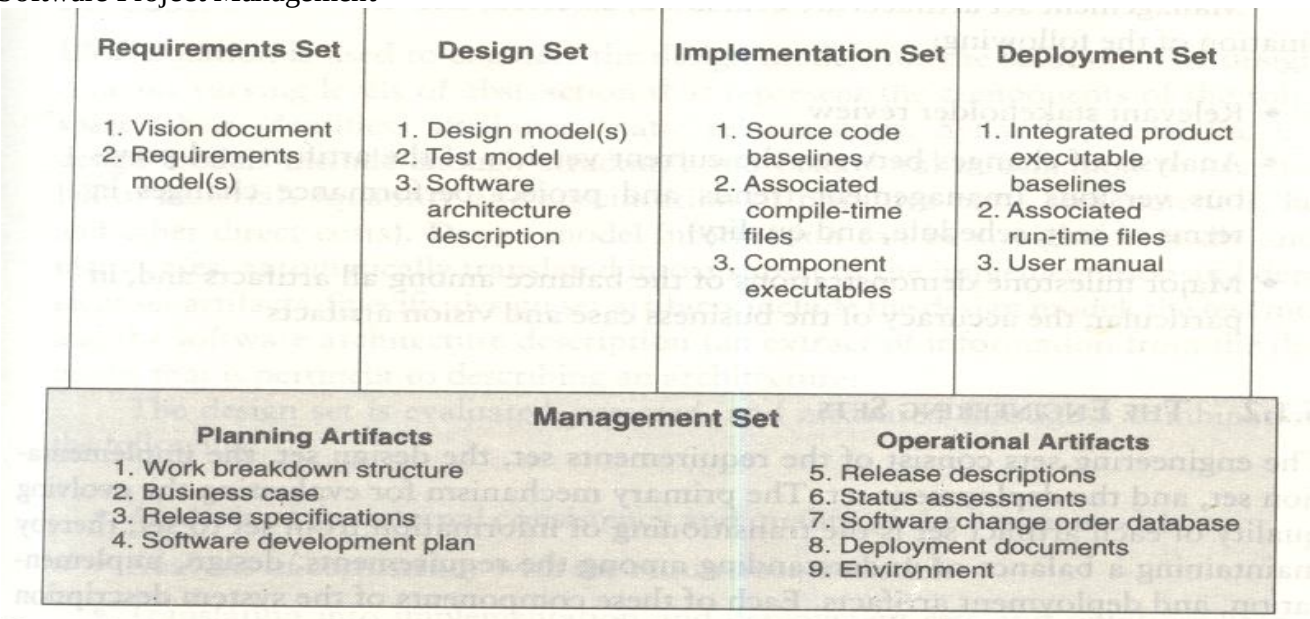


FIGURE 6-1. *Overview of the artifact sets*

6.1.1 THE MANAGEMENT SET

The management set captures the artifacts associated with process planning and execution. These artifacts use ad hoc notations, including text, graphics, or whatever representation is required to capture the "contracts" among project personnel (project management, architects, developers, testers, marketers, administrators), among stakeholders (funding authority, user, software project manager, organization manager, regulatory agency), and between project personnel and stakeholders. Specific artifacts included in this set are the work breakdown structure (activity breakdown and financial tracking mechanism), the business case (cost, schedule, profit expectations), the release specifications (scope, plan, objectives for release baselines), the software development plan (project process instance), the release descriptions (results of release baselines), the status assessments (periodic snapshots of project progress), the software change orders (descriptions of discrete baseline changes), the deployment documents (cutover plan, training course, sales rollout kit), and the environment (hardware and software tools, process automation, & documentation).

Management set artifacts are evaluated, assessed, and measured through a combination of the following:

- Relevant stakeholder review
- Analysis of changes between the current version of the artifact and previous versions
- Major milestone demonstrations of the balance among all artifacts and, in particular, the accuracy of the business case and vision artifacts

6.1.2 THE ENGINEERING SETS

The engineering sets consist of the requirements set, the design set, the implementation set, and the deployment set.

Requirements Set

Requirements artifacts are evaluated, assessed, and measured through a combination of the following:

- Analysis of consistency with the release specifications of the management set
- Analysis of consistency between the vision and the requirements models
- Mapping against the design, implementation, and deployment sets to evaluate the consistency and completeness and the semantic balance between information in the different sets

- Analysis of changes between the current version of requirements artifacts and previous versions (scrap, rework, and defect elimination trends)
- Subjective review of other dimensions of quality

Design Set

UML notation is used to engineer the design models for the solution. The design set contains varying levels of abstraction that represent the components of the solution space (their identities, attributes, static relationships, dynamic interactions). The design set is evaluated, assessed, and measured through a combination of the following:

- Analysis of the internal consistency and quality of the design model
- Analysis of consistency with the requirements models
- Translation into implementation and deployment sets and notations (for example, traceability, source code generation, compilation, linking) to evaluate the consistency and completeness and the semantic balance between information in the sets
- Analysis of changes between the current version of the design model and previous versions (scrap, rework, and defect elimination trends)
- Subjective review of other dimensions of quality

Implementation set

The implementation set includes source code (programming language notations) that represents the tangible implementations of components (their form, interface, and dependency relationships)

Implementation sets are human-readable formats that are evaluated, assessed, and measured through a combination of the following:

- Analysis of consistency with the design models
- Translation into deployment set notations (for example, compilation and linking) to evaluate the consistency and completeness among artifact sets
- Assessment of component source or executable files against relevant evaluation criteria through inspection, analysis, demonstration, or testing
- Execution of stand-alone component test cases that automatically compare expected results with actual results
- Analysis of changes between the current version of the implementation set and previous versions (scrap, rework, and defect elimination trends)
- Subjective review of other dimensions of quality

Deployment Set

The deployment set includes user deliverables and machine language notations, executable software, and the build scripts, installation scripts, and executable target specific data necessary to use the product in its target environment.

Deployment sets are evaluated, assessed, and measured through a combination of the following:

- Testing against the usage scenarios and quality attributes defined in the requirements set to evaluate the consistency and completeness and the~ semantic balance between information in the two sets
- Testing the partitioning, replication, and allocation strategies in mapping components of the implementation set to physical resources of the deployment system (platform type, number, network topology)
- Testing against the defined usage scenarios in the user manual such as installation, user-oriented

dynamic reconfiguration, mainstream usage, and anomaly management

- Analysis of changes between the current version of the deployment set and previous versions (defect elimination trends, performance changes)
- Subjective review of other dimensions of quality

Each artifact set is the predominant development focus of one phase of the life cycle; the other sets take on check and balance roles. As illustrated in Figure 6-2, each phase has a predominant focus: Requirements are the focus of the inception phase; design, the elaboration phase; implementation, the construction phase; and deployment, the transition phase. The management artifacts also evolve, but at a fairly constant level across the life cycle.

Most of today's software development tools map closely to one of the five artifact sets.

1. Management: scheduling, workflow, defect tracking, change management, documentation, spreadsheet, resource management, and presentation tools
2. Requirements: requirements management tools
3. Design: visual modeling tools
4. Implementation: compiler/debugger tools, code analysis tools, test coverage analysis tools, and test management tools
5. Deployment: test coverage and test automation tools, network management tools, commercial components (operating systems, GUIs, RDBMS, networks, middleware), and installation tools.

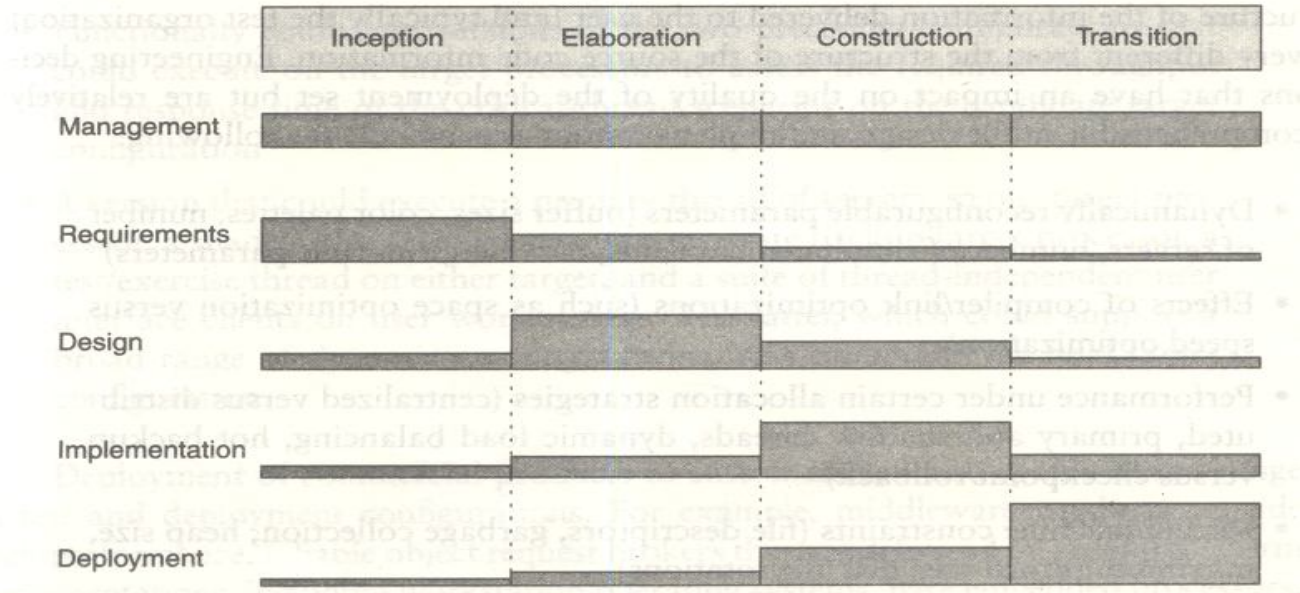


FIGURE 6-2. *Life-cycle focus on artifact sets*

Implementation Set versus Deployment Set

The separation of the implementation set (source code) from the deployment set (executable code) is important because there are very different concerns with each set. The structure of the information delivered to the user (and typically the test organization) is very different from the structure of the source code information. Engineering decisions that have an impact on the quality of the deployment set but are relatively incomprehensible in the design and implementation sets include the following:

- Dynamically reconfigurable parameters (buffer sizes, color palettes, number of servers, number of simultaneous clients, data files, run-time parameters)
- Effects of compiler/link optimizations (such as space optimization versus speed optimization)
- Performance under certain allocation strategies (centralized versus distributed, primary and shadow threads, dynamic load balancing, hot backup versus checkpoint/rollback)

- Virtual machine constraints (file descriptors, garbage collection, heap size, maximum record size, disk file rotations)
- Process-level concurrency issues (deadlock and race conditions)
- Platform-specific differences in performance or behavior

6.1.3 ARTIFACT EVOLUTION OVER THE LIFE CYCLE

Each state of development represents a certain amount of precision in the final system description. Early in the life cycle, precision is low and the representation is generally high. Eventually, the precision of representation is high and everything is specified in full detail. Each phase of development focuses on a particular artifact set. At the end of each phase, the overall system state will have progressed on all sets, as illustrated in Figure 6-3.

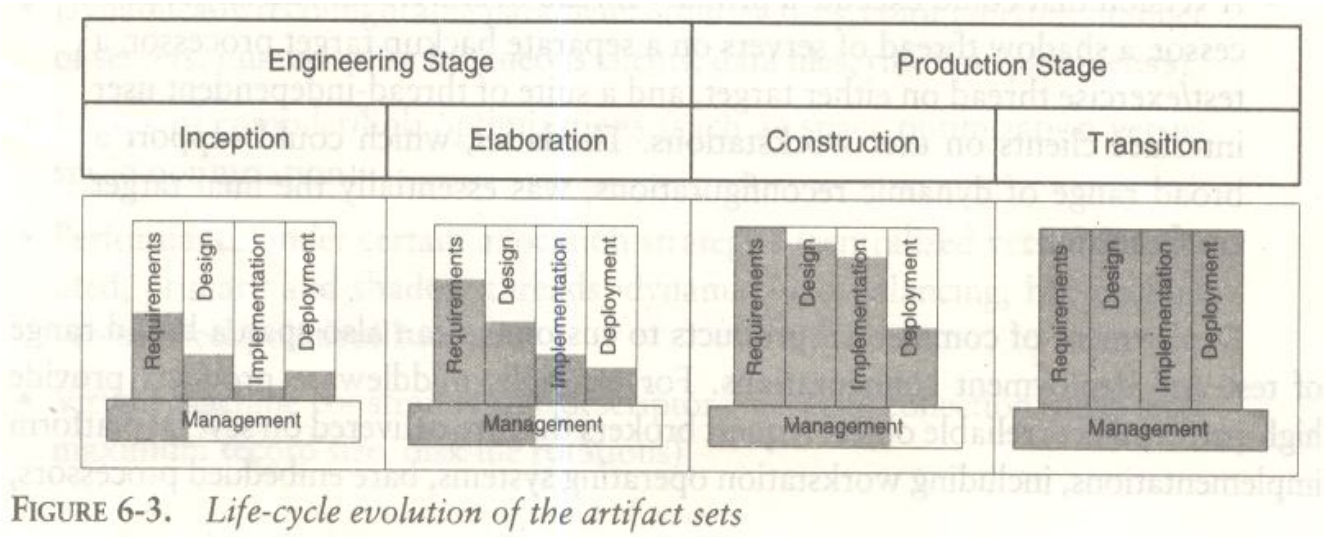


FIGURE 6-3. *Life-cycle evolution of the artifact sets*

The **inception** phase focuses mainly on critical requirements usually with a secondary focus on an initial deployment view. During the **elaboration phase**, there is much greater depth in requirements, much more breadth in the design set, and further work on implementation and deployment issues. The main focus of the **construction** phase is design and implementation. The main focus of the **transition** phase is on achieving consistency and completeness of the deployment set in the context of the other sets.

6.1.4 TEST ARTIFACTS

- The test artifacts must be developed concurrently with the product from inception through deployment. Thus, testing is a full-life-cycle activity, not a late life-cycle activity.
- The test artifacts are communicated, engineered, and developed within the same artifact sets as the developed product.
- The test artifacts are implemented in programmable and repeatable formats (as software programs).
- The test artifacts are documented in the same way that the product is documented.
- Developers of the test artifacts use the same tools, techniques, and training as the software engineers developing the product.

Test artifact subsets are highly project-specific, the following example clarifies the relationship between test artifacts and the other artifact sets. Consider a project to perform seismic data processing for the purpose of

oil exploration. This system has three fundamental subsystems: (1) a sensor subsystem that captures raw seismic data in real time and delivers these data to (2) a technical operations subsystem that converts raw data into an organized database and manages queries to this database from (3) a display subsystem that allows workstation operators to examine seismic data in human-readable form. Such a system would result in the following test artifacts:

- Management set. The release specifications and release descriptions capture the objectives, evaluation criteria, and results of an intermediate milestone. These artifacts are the test plans and test results negotiated among internal project teams. The software change orders capture test results (defects, testability changes, requirements ambiguities, enhancements) and the closure criteria associated with making a discrete change to a baseline.
- Requirements set. The system-level use cases capture the operational concept for the system and the acceptance test case descriptions, including the expected behavior of the system and its quality attributes. The entire requirement set is a test artifact because it is the basis of all assessment activities across the life cycle.
- Design set. A test model for nondeliverable components needed to test the product baselines is captured in the design set. These components include such design set artifacts as a seismic event simulation for creating realistic sensor data; a "virtual operator" that can support unattended, after-hours test cases; specific instrumentation suites for early demonstration of resource usage; transaction rates or response times; and use case test drivers and component stand-alone test drivers.
- Implementation set. Self-documenting source code representations for test components and test drivers provide the equivalent of test procedures and test scripts. These source files may also include human-readable data files representing certain statically defined data sets that are explicit test source files. Output files from test drivers provide the equivalent of test reports.
- Deployment set. Executable versions of test components, test drivers, and data files are provided.

6.2 MANAGEMENT ARTIFACTS

The management set includes several artifacts that capture intermediate results and ancillary information necessary to document the product/process legacy, maintain the product, improve the product, and improve the process.

Business Case

The business case artifact provides all the information necessary to determine whether the project is worth investing in. It details the expected revenue, expected cost, technical and management plans, and backup data necessary to demonstrate the risks and realism of the plans. The main purpose is to transform the vision into economic terms so that an organization can make an accurate ROI assessment. The financial forecasts are evolutionary, updated with more accurate forecasts as the life cycle progresses. Figure 6-4 provides a default outline for a business case.

Software Development Plan

The software development plan (SDP) elaborates the process framework into a fully detailed plan. Two indications of a useful SDP are periodic updating (it is not stagnant shelfware) and understanding and acceptance by managers and practitioners alike. Figure 6-5 provides a default outline for a software development plan.

I. **Context (domain, market, scope)**
II. **Technical approach**
   A. Feature set achievement plan
   B. Quality achievement plan
   C. Engineering trade-offs and technical risks
III. **Management approach**
   A. Schedule and schedule risk assessment
   B. Objective measures of success
IV. **Evolutionary appendixes**
   A. Financial forecast
      1. Cost estimate
      2. Revenue estimate
      3. Bases of estimates

FIGURE 6-4. *Typical business case outline*

I. **Context (scope, objectives)**
II. **Software development process**
   A. Project primitives
      1. Life-cycle phases
      2. Artifacts
      3. Workflows
      4. Checkpoints
   B. Major milestone scope and content
   C. Process improvement procedures
III. **Software engineering environment**
   A. Process automation (hardware and software resource configuration)
   B. Resource allocation procedures (sharing across organizations, security access)
IV. **Software change management**
   A. Configuration control board plan and procedures
   B. Software change order definitions and procedures
   C. Configuration baseline definitions and procedures
V. **Software assessment**
   A. Metrics collection and reporting procedures
   B. Risk management procedures (risk identification, tracking, and resolution)
   C. Status assessment plan
   D. Acceptance test plan
VI. **Standards and procedures**
   A. Standards and procedures for technical artifacts
VII. **Evolutionary appendixes**
   A. Minor milestone scope and content
   B. Human resources (organization, staffing plan, training plan)

FIGURE 6-5. *Typical software development plan outline*

Work Breakdown Structure

Work breakdown structure (WBS) is the vehicle for budgeting and collecting costs. To monitor and control a project's financial performance, the software project man1ger must have insight into project costs and how they are expended. The structure of cost accountability is a serious project planning constraint.

Software Change Order Database

Managing change is one of the fundamental primitives of an iterative development process. With greater change freedom, a project can iterate more productively. This flexibility increases the content, quality, and number of iterations that a project can achieve within a given schedule. Change freedom has been achieved in practice through automation, and today's iterative development environments carry the burden of change management. Organizational processes that depend on manual change management techniques have encountered major inefficiencies.

Release Specifications

The scope, plan, and objective evaluation criteria for each baseline release are derived from the vision statement as well as many other sources (make/buy analyses, risk management concerns, architectural considerations, shots in the dark, implementation constraints, quality thresholds). These artifacts are intended to evolve along with the process, achieving greater fidelity as the life cycle progresses and requirements understanding matures. Figure 6-6 provides a default outline for a release specification

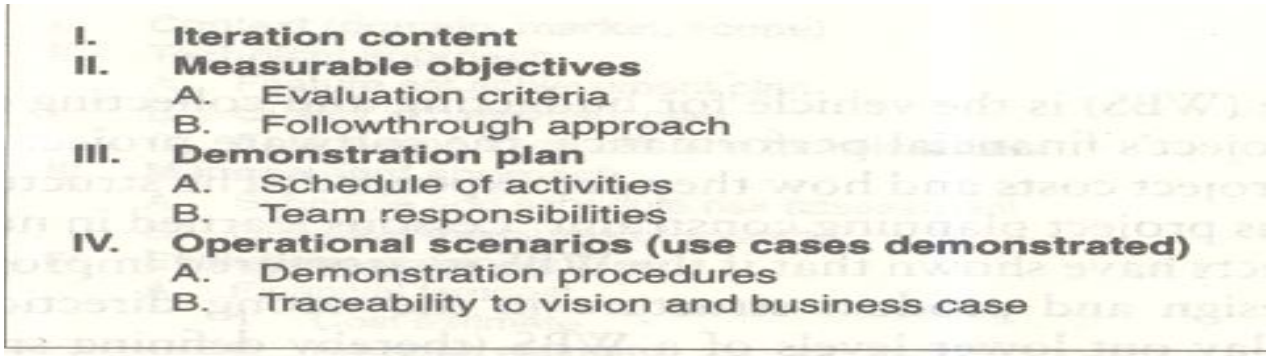


FIGURE 6-6. *Typical release specification outline*

Release Descriptions

Release description documents describe the results of each release, including performance against each of the evaluation criteria in the corresponding release specification. Release baselines should be accompanied by a release description document that describes the evaluation criteria for that configuration baseline and provides substantiation (through demonstration, testing, inspection, or analysis) that each criterion has been addressed in an acceptable manner. Figure 6-7 provides a default outline for a release description.

Status Assessments

Status assessments provide periodic snapshots of project health and status, including the software project manager's risk assessment, quality indicators, and management indicators. Typical status assessments should include a review of resources, personnel staffing, financial data (cost and revenue), top 10 risks, technical progress (metrics snapshots), major milestone plans and results, total project or product scope & action items

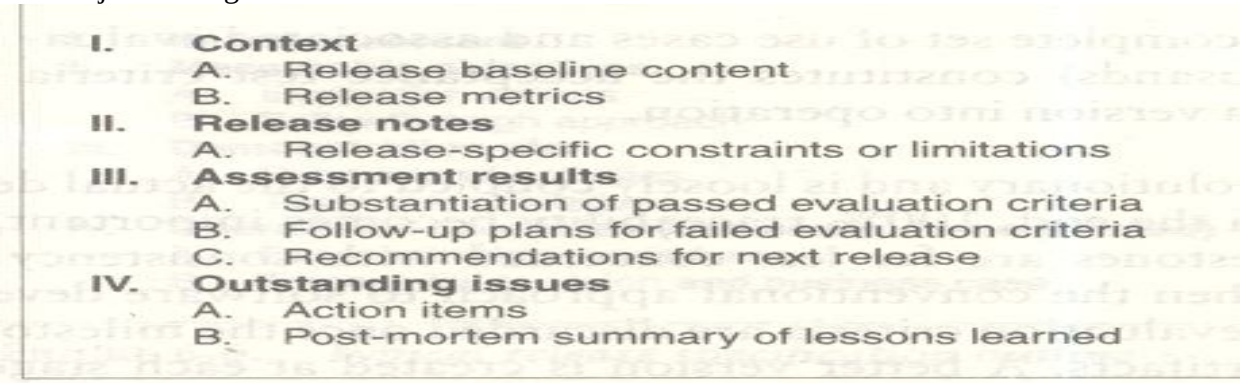


FIGURE 6-7. *Typical release description outline*

Environment

An important emphasis of a modern approach is to define the development and maintenance environment as a first-class artifact of the process. A robust, integrated development environment must support automation of the development process. This environment should include requirements management, visual modeling, document automation, host and target programming tools, automated regression testing, and continuous and integrated change management, and feature and defect tracking.

Deployment

A deployment document can take many forms. Depending on the project, it could include several document subsets for transitioning the product into operational status. In big contractual efforts in which the system is delivered to a separate maintenance organization, deployment artifacts may include computer system operations manuals, software installation manuals, plans and procedures for cutover (from a legacy system), site surveys, and so forth. For commercial software products, deployment artifacts may include marketing plans, sales rollout kits, and training courses.

Management Artifact Sequences

In each phase of the life cycle, new artifacts are produced and previously developed artifacts are updated to incorporate lessons learned and to capture further depth and breadth of the solution. Figure 6-8 identifies a typical sequence of artifacts across the life-cycle phases.

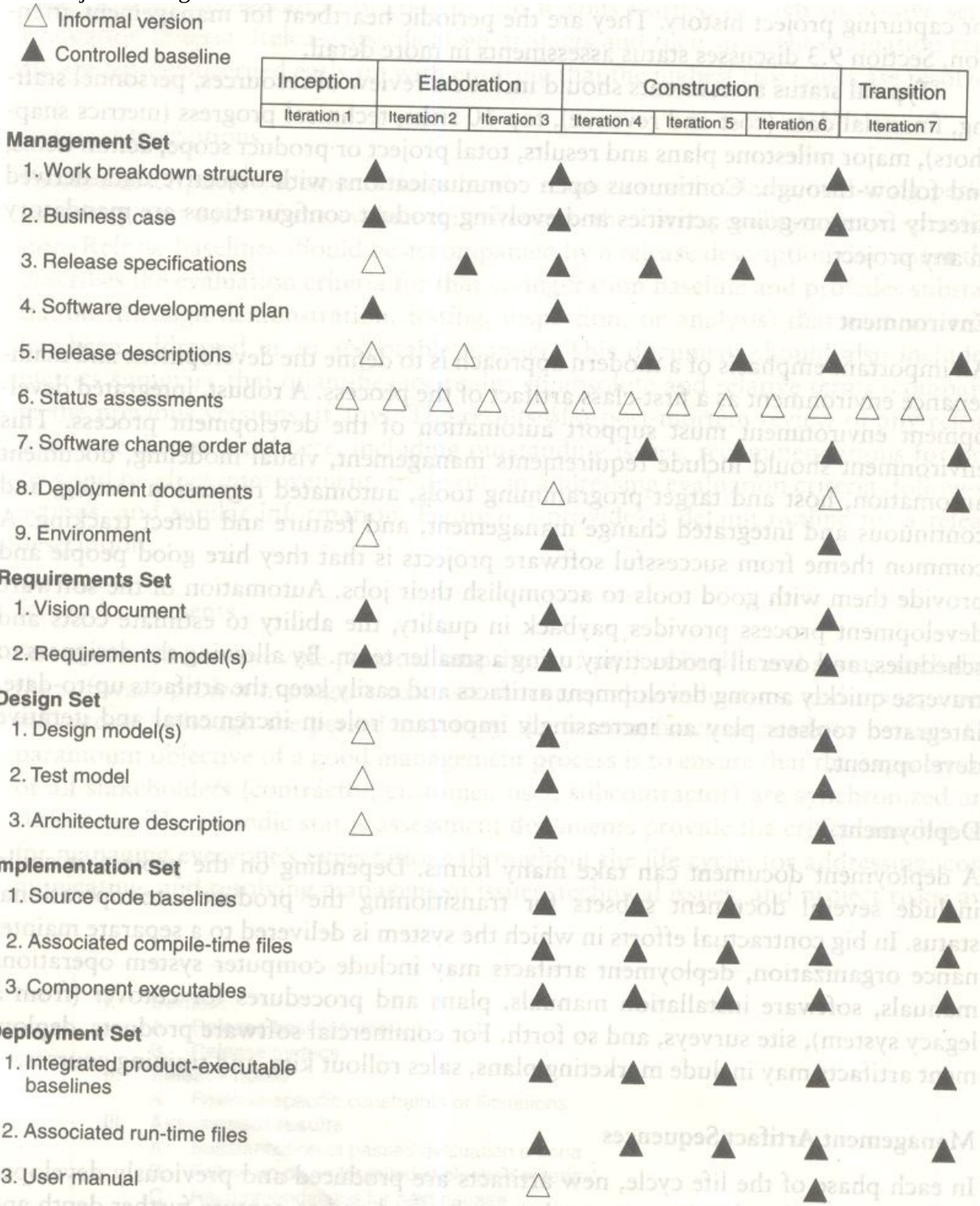


FIGURE 6-8. *Artifact sequences across a typical life cycle*

6.3 ENGINEERING ARTIFACTS

Most of the engineering artifacts are captured in rigorous engineering notations such as UML, programming languages, or executable machine codes. Three engineering artifacts are explicitly intended for more general review, and they deserve further elaboration.

Vision Document

The vision document provides a complete vision for the software system under development and. supports the contract between the funding authority and the development organization. A project vision is meant to be changeable as understanding evolves of the requirements, architecture, plans, and technology. A good vision document should change slowly. Figure 6-9 provides a default outline for a vision document.

I. Feature set description
- A. Precedence and priority

II. Quality attributes and ranges

III. Required constraints
- A. External interfaces

IV. Evolutionary appendixes
- A. Use cases
  1. Primary scenarios
  2. Acceptance criteria and tolerances
- B. Desired freedoms (potential change scenarios)

FIGURE 6-9. *Typical vision document outline*

Architecture Description

The architecture description provides an organized view of the software architecture under development. It is extracted largely from the design model and includes views of the design, implementation, and deployment sets sufficient to understand how the operational concept of the requirements set will be achieved. The breadth of the architecture description will vary from project to project depending on many factors. Figure 6-10 provides a default outline for an architecture description.

I. Architecture overview
- A. Objectives
- B. Constraints
- C. Freedoms

II. Architecture views
- A. Design view
- B. Process view
- C. Component view
- D. Deployment view

III. Architectural interactions
- A. Operational concept under primary scenarios
- B. Operational concept under secondary scenarios
- C. Operational concept under anomalous conditions

IV. Architecture performance

V. Rationale, trade-offs, and other substantiation

FIGURE 6-10. *Typical architecture description outline*

Software User Manual

The software user manual provides the user with the reference documentation necessary to support the delivered software. Although content is highly variable across application domains, the user manual should include installation procedures, usage procedures and guidance, operational constraints, and a user interface description, at a minimum. For software products with a user interface, this manual should be developed early in the life cycle because it is a necessary mechanism for communicating and stabilizing an important subset of requirements. The user manual should be written by members of the test team, who are more likely to understand the user's perspective than the development team.

6.4 PRAGMATIC ARTIFACTS

- **People want to review information but don't understand the language of the artifact**. Many interested reviewers of a particular artifact will resist having to learn the engineering language in which the artifact is written. It is not uncommon to find people (such as veteran software managers, veteran quality assurance specialists, or an auditing authority from a regulatory agency) who react as follows: "I'm not going to learn UML, but I want to review the design of this software, so give me a separate description such as some flowcharts and text that I can understand."
- **People want to review the information but don't have access to the tools.** It is not very common for the development organization to be fully tooled; it is extremely rare that the/other stakeholders have any capability to review the engineering artifacts on-line. Consequently, organizations are forced to exchange paper documents. Standardized formats (such as UML, spreadsheets, Visual Basic, C++, and Ada 95), visualization tools, and the Web are rapidly making it economically feasible for all stakeholders to exchange information electronically.
- **Human-readable engineering artifacts should use rigorous notations that are complete, consistent, and used in a self-documenting manner.** Properly spelled English words should be used for all identifiers and descriptions. Acronyms and abbreviations should be used only where they are well accepted jargon in the context of the component's usage. Readability should be emphasized and the use of proper English words should be required in all engineering artifacts. This practice enables understandable representations, browse able formats (paperless review), more-rigorous notations, and reduced error rates.
- **Useful documentation is self-defining: It is documentation that gets used.**
- **Paper is tangible; electronic artifacts are too easy to change**. On-line and Web-based artifacts can be changed easily and are viewed with more skepticism because of their inherent volatility.

Unit – III Important questions

1.	**Explain briefly two stages of the life cycle engineering and production.**
2.	**Explain different phases of the life cycle process?**
3.	**Explain the goal of Inception phase, Elaboration phase, Construction phase and Transition phase.**
4.	**Explain the overview of the artifact set**
5.	**Write a short note on** **(a) Management Artifacts (b) Engineering Artifacts (c) Pragmatic Artifacts**

Model-Based Software Architectures

INTRODUCTION:

Software architecture is the central design problem of a complex software system in the same way an *architecture is the software system design*.

--The ultimate goal of the engineering stage is to converge on a stable architecture baseline.

--Architecture is not a paper document. It is a collection of information across all the engineering sets.

--Architectures are described by extracting the essential information from the design models.

--A *model is a relatively independent abstraction of a system*.

--A *view is a subset of a model that abstracts a specific, relevant perspective*

ARCHITECTURE : A MANAGEMENT PERSPECTIVE

--The most critical and technical product of a software project is its architecture

-- If a software development team is to be successful, the interproject communications, as captured in software architecture, must be accurate and precise.

From the management point of view, three different aspects of architecture

1. An *architecture (the intangible design concept) is the design of software* system it includes all engineering necessary to specify a complete bill of materials. Significant make or buy decisions resolved, and all custom components are elaborated so that individual component costs and construction/assembly costs can be determined with confidence.

- 2. An *architecture baseline (the tangible artifacts) is a slice of information across* the engineering artifact sets sufficient to satisfy all stakeholders that the vision (function and quality) can be achieved within the parameters of the business case (cost, profit, time, technology, people).
- 3. An *architectural description is an organized subset of information extracted* from the design set model's. It explains how the intangible concept is realized in the tangible artifacts. The number of views and level of detail in each view can vary widely. For example the architecture of the software architecture of a small development tool.

There is a close relationship between software architecture and the modern software development process because of the following reasons:

1. A stable software architecture is nothing but a project milestone where critical make/buy decisions should have been resolved. The life-cycle represents a transition from the engineering stage of a project to the production stage.
2. Architecture representation provide a basis for balancing the trade-offs between the problem space (requirements and constraints) and the solution space (the operational product).
3. The architecture and process encapsulate many of the important communications among individuals, teams, organizations, and stakeholders.
4. Poor architecture and immature process are often given as reasons for project failure.
5. In order to proper planning, a mature process, understanding the primary requirements and demonstrable architecture are important fundamentals.
6. Architecture development and process definition are the intellectual steps that map the problem to a solution without violating the constraints; they require human innovation and cannot be automated.

ARCHITECTURE: A TECHNICAL PERSPECTIVE

--Software architecture include the structure of software systems, their behavior, and the patterns that guide these elements, their collaborations, and their composition.

--An architecture framework is defined in terms of views is the abstraction of the UML models in the design set. Where as architecture view is an abstraction of the design model, include fullbreadth and depth of information.

Most real-world systems require four types of views:

1) Design: describes architecturally significant structures and functions of the design model.
2) Process: describes concurrency and control thread relationships among the design, component, and deployment views.
3) Component: describes the structure of the implementation set.
4) Deployment: describes the structure of the deployment set.

The deployment model provides for visualization of the deployment set.

1. The *use case view describes how the system s critical use cases are realized by* elements of the design model. It is modeled statistically by using use case diagrams, and dynamically by using any of the UML behavioral diagrams.
2. The *design view describes the architecturally significant elements of the design* model. It is modeled statistically by using class and object diagrams, and dynamically using any of the UML behavioral diagrams.
3. The *process view addresses the run-time collaboration issues involved in* executing the architecture on a distributed deployment model, including logical software topology, inter process communication, and state mgmt. it is modeled statistically using deployment diagrams, and dynamically using any of the UML behavioral diagrams.

4. The *component view describes the architecturally significant elements of the* implementation set. It is modeled statistically using component diagrams, and dynamically using any of the UML behavioral diagrams.

5.The *deployment view addresses the executable realization of the system, including* the allocation of logical processes in the distributed view to physical resources of the deployment network. It is modeled statistically using deployment diagrams,and dynamically using any of UML behavioral diagrams.

- An architecture baseline is defined as a balanced subset of information across all sets, where as an architecture description is completely encapsulated within the design set.

Generally architecture base line include:

1) Requirements 2) Design

3) Implementation 4) Deployment